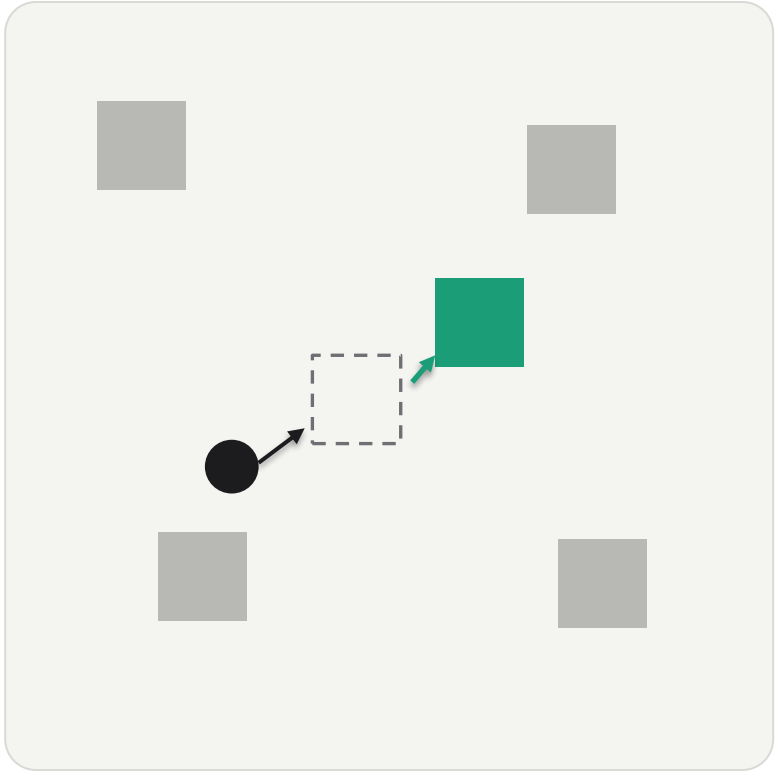
WORLDS WORKSHOP · IEEE/RSJ IROS 2026

Modeling What Changes

Sparse, Residual World Models for Object-Centric Manipulation



Param Thakkar Veermata Jijabai Technological Institute

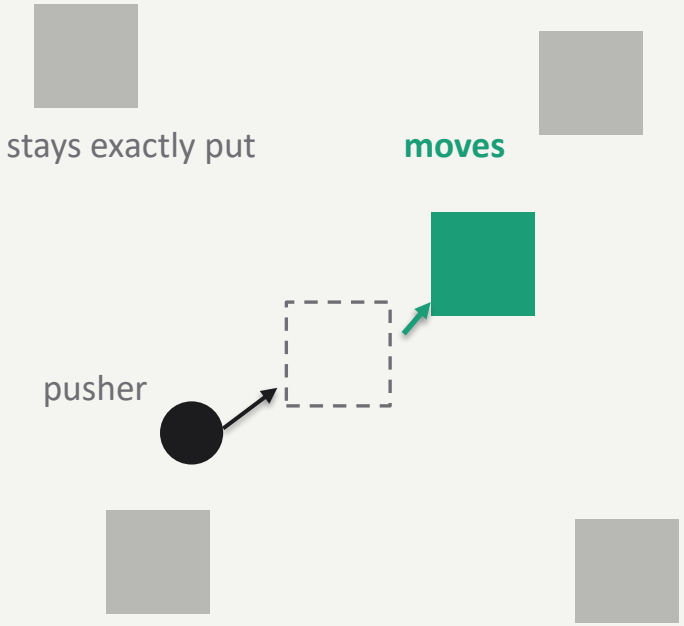
Parsika Paresh Shah Arizona State University

Manisha Sushant Gote ZuiGO Private Limited

Code, data generators, checkpoints: https://github.com/ParamThakkar123/sparse_world_models

THE OBSERVATION

Physical scenes are sparse in change



One push. One object moves.

Every other object is exactly where it was.

- **Dominant recipe:** encode the scene, regress the *entire* next state every step
- It has to re-produce the static majority **at every step, for every object**
- The change structure of the world is **thrown away** before learning starts

THE PROBLEM

Re-predicting everything has three costs

1

Capacity

Parameters and operations are spent on the static majority, not on the few objects that matter.

2

Error injection

A regressor that outputs every pose perturbs objects it should have copied. Closed loop, that error accumulates on objects that never moved: the predicted world drifts.

3

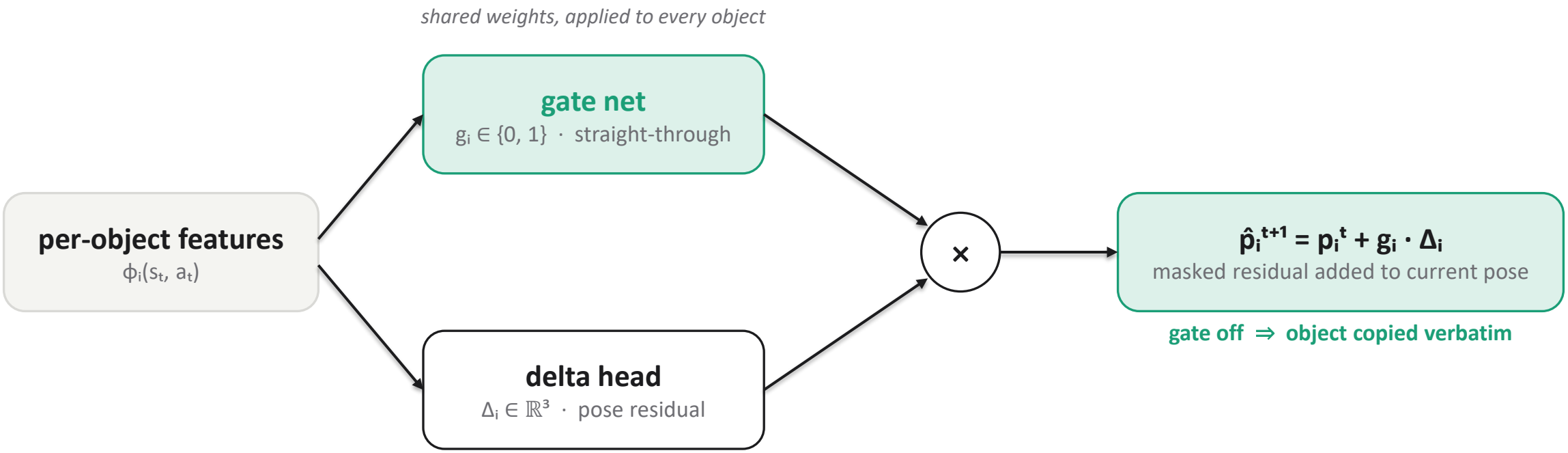
No handle on “what”

A monolithic predictor never says which objects it expects to move. For control and debugging, that belief is entangled in one regression output.

The second cost is the insidious one: it is exactly the objects that never moved that drift.

OUR MODEL

Gate what changes, copy the rest



- **Per-object gate** decides which objects move; a **residual delta head** predicts a pose increment applied only to gated objects
- **Hard at inference:** exactly zero update for un-gated objects. Trained end-to-end with a Gumbel straight-through gate
- **Count-agnostic:** heads are shared across objects and no layer is sized to N, so one model runs at any object count

TRAINING

One loss, three terms

$$\mathcal{L} = \mathrm{BCE}(g, m^*) + \lambda_\Delta \cdot m^* \cdot \|\Delta - \Delta^*\|^2 + \lambda_s \cdot \bar{g}$$

Change detection

Class-balanced binary cross-entropy against the ground-truth changed mask m*. This is what the gate learns from.

Residual regression

L2 on the delta, supervised only on objects that actually changed. Un-gated objects contribute nothing.

Sparsity

Penalty on the mean gate activation ḡ. Trades recall for precision: λs 0 → 1 moves precision 0.94 → 0.97, recall 0.81 → 0.71.

Small by design: gate and delta heads are 2-layer, width-128 MLPs. Under 0.1M parameters, trains and plans on a laptop CPU.

SETUP

A controlled tabletop pushing benchmark

- **Environment** procedural MuJoCo tabletop, N ∈ {3, 5, 8} free 5 cm boxes, scripted pushing policy
- **State** pusher position, per-object planar pose (x, y, θ) and velocity, goal. Structured state, not pixels
- **Action** end-effector Δxy to a position-controlled pusher
- **Data** 250 episodes × 100 steps per (N, seed), filtered to a hard subset with real motion; 80/10/10 split with a configuration-leakage guard
- **Seeds** headline tables are mean ± std over 3 training seeds

BASELINES

Dense MLP regresses all poses; width 256, 3 layers

No-op predicts no change at all. The floor every world model must beat

METRICS

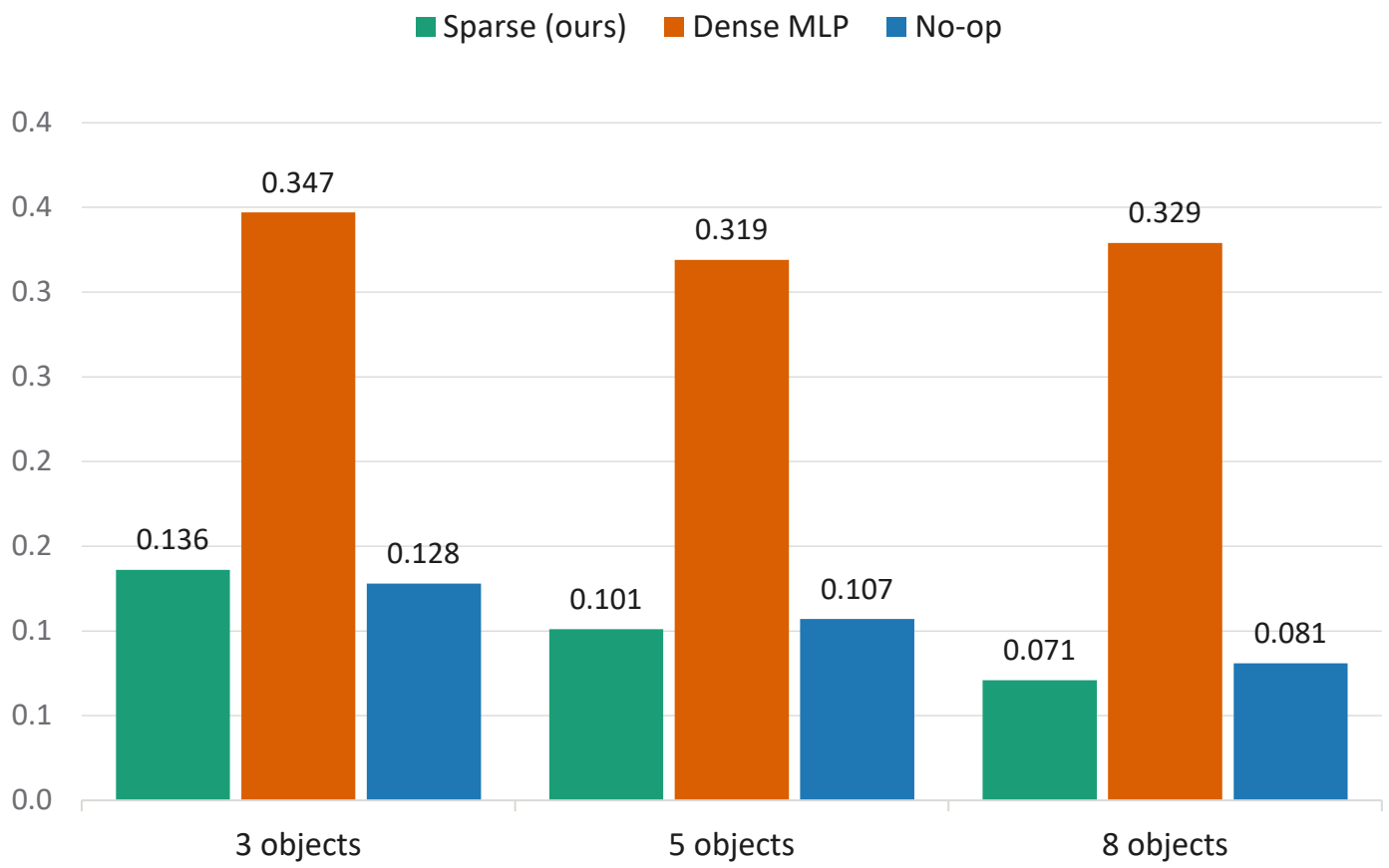
Per-object pose L2, **split into changed and unchanged objects**

Change-detection precision, recall, F1

Parameters, operations, latency

PREDICTION ACCURACY

Lower error at a fraction of the parameters



Overall per-object pose L2 on held-out transitions, lower is better (Tables I and II; 3 seeds).

2.5 – 4.6×

lower overall L2 than dense, at every N

8.6 – 11.1×

fewer parameters (6.9k–10.8k vs 77k–92k)

F1 0.80 – 0.87

change detection, precision 0.92–0.97; dense is degenerate (recall ≈ 1)

PREDICTION ACCURACY

Sparse moves only what moves

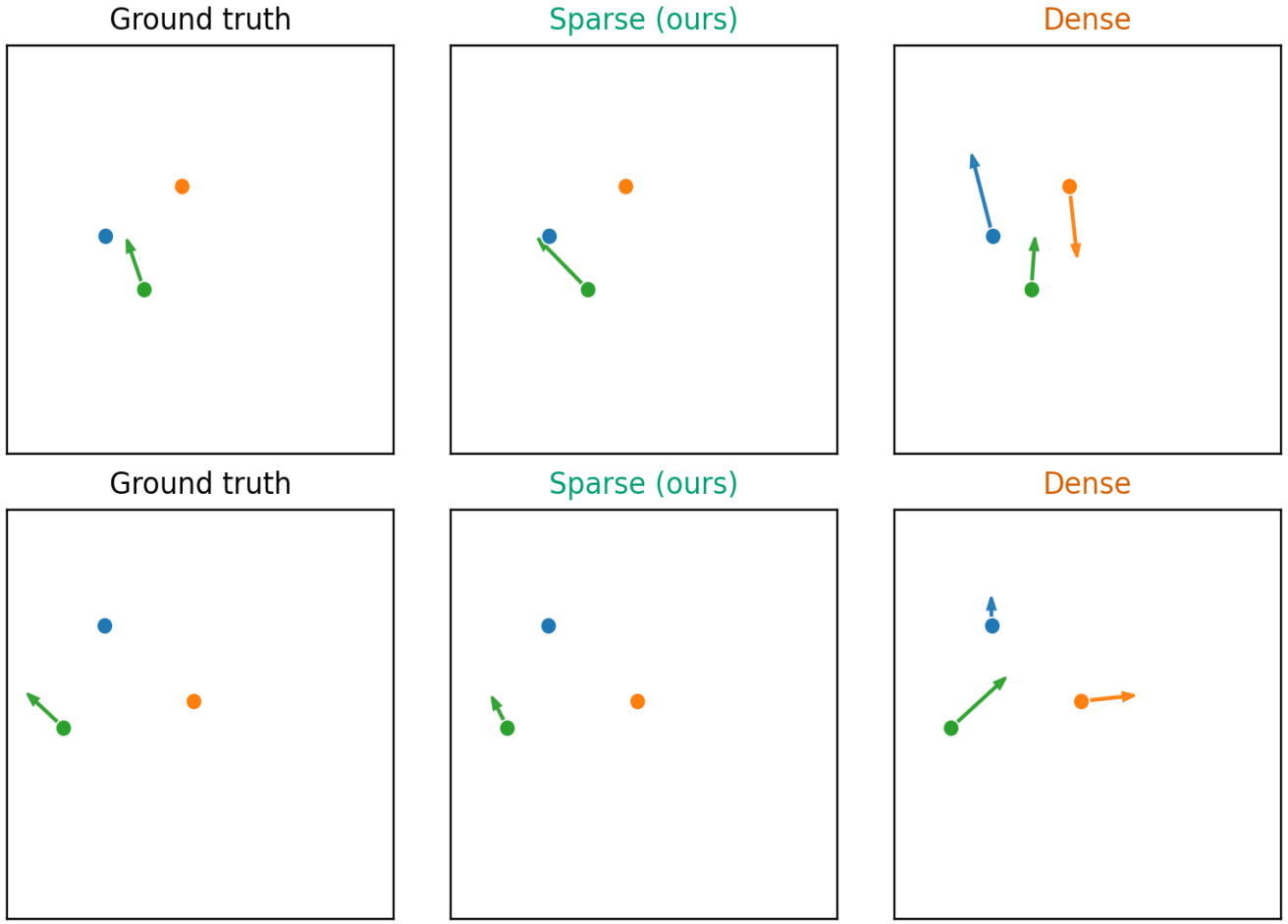
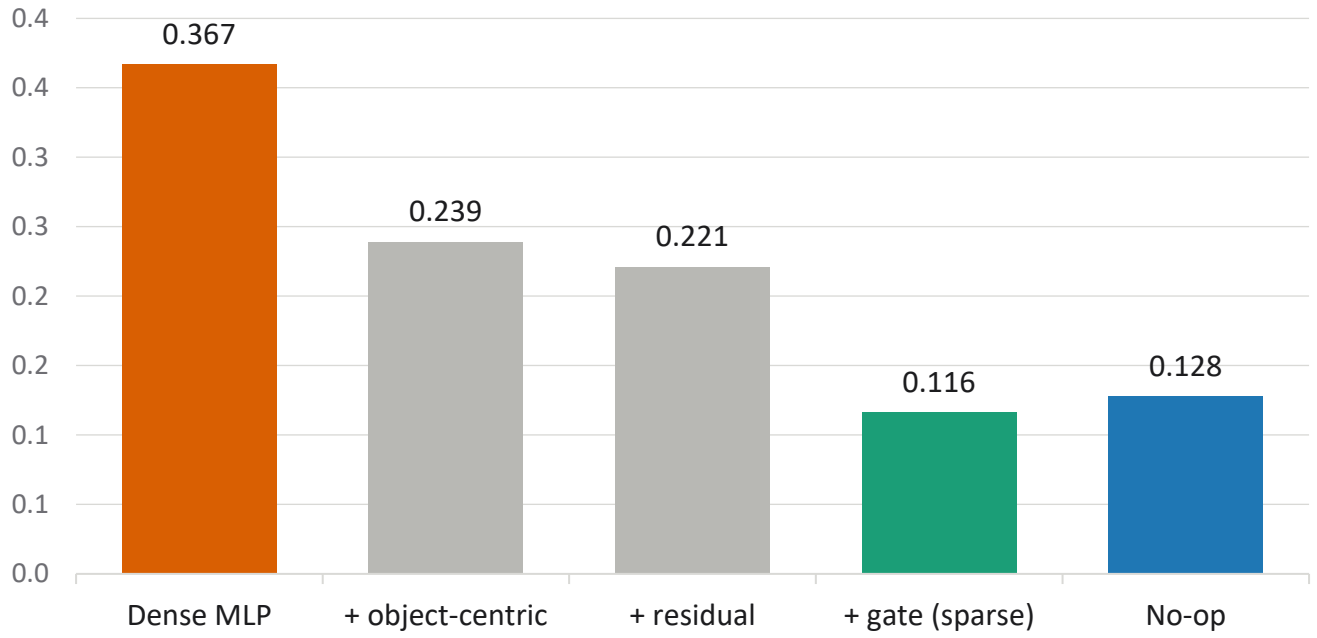


Fig. 2 of the paper, two held-out scenes.

- **Arrows** are predicted displacement, current to next position
- **Ground truth:** one object moves per step
- **Sparse (ours):** the gate fires on that object alone; the other two are copied
- **Dense:** hallucinates motion on every object, including the ones that should stay put

WHY IT WINS

Where the win actually comes from



Capacity-matched ladder, 3 objects, seed 0 (Table II). Each rung adds one ingredient.

Unchanged-object L2 down the ladder: 0.211 → 0.111 → 0.086 → **0.001**

The gate is the largest single step (1.7–2.0× at every N) and the only rung that reaches below no-op.

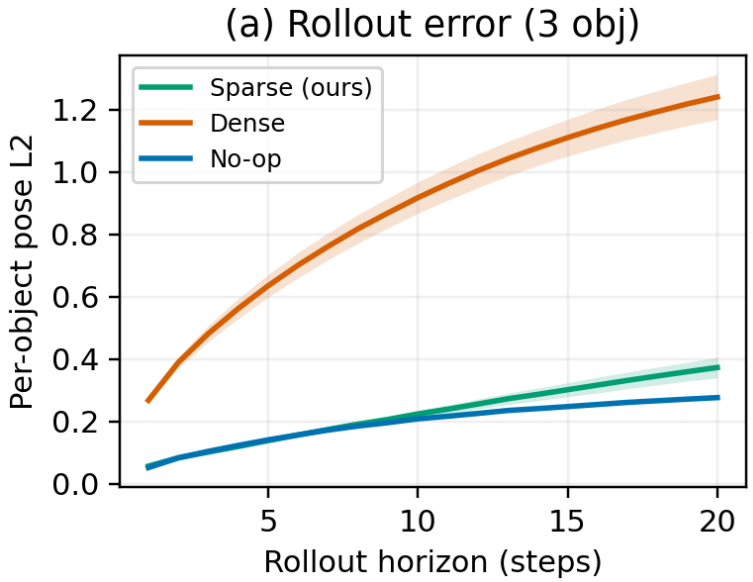
Changed-object L2 with a perfect gate

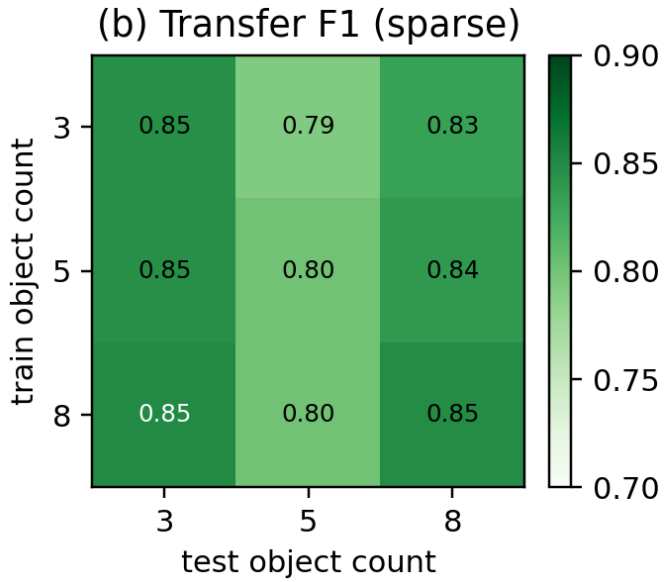
N	predicted gate	oracle gate	no-op
3	0.312	0.314	0.348
5	0.426	0.432	0.446
8	0.466	0.474	0.470

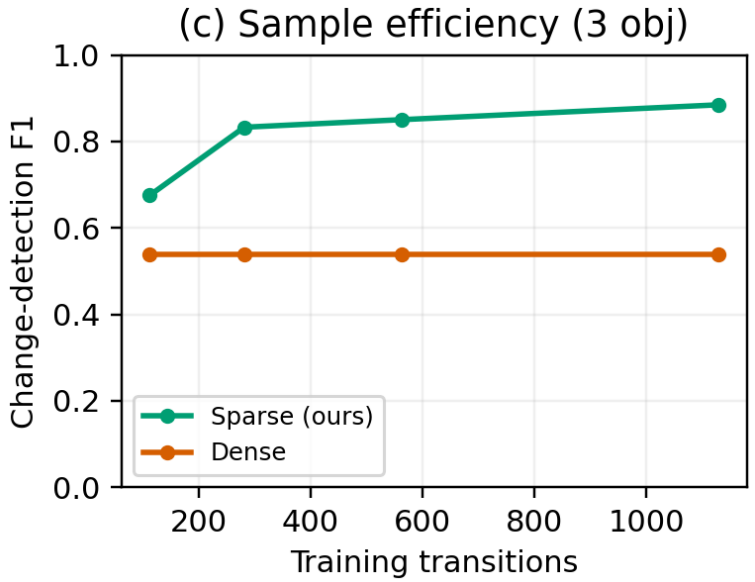
Feeding the delta head the ground-truth mask barely moves the number.

So the win is detecting what changed and **not corrupting the static majority**. On the movers themselves, sparse is at roughly no-op quality: the bottleneck is delta regression, not detection.

Rollout, transfer across counts, sample efficiency







(a) Rollout, 20 steps closed loop

Sparse compounds 3.4–6.4× less error than dense at every N, but stays above no-op overall: it wins on static objects and only ties on movers.

(b) Zero-shot transfer

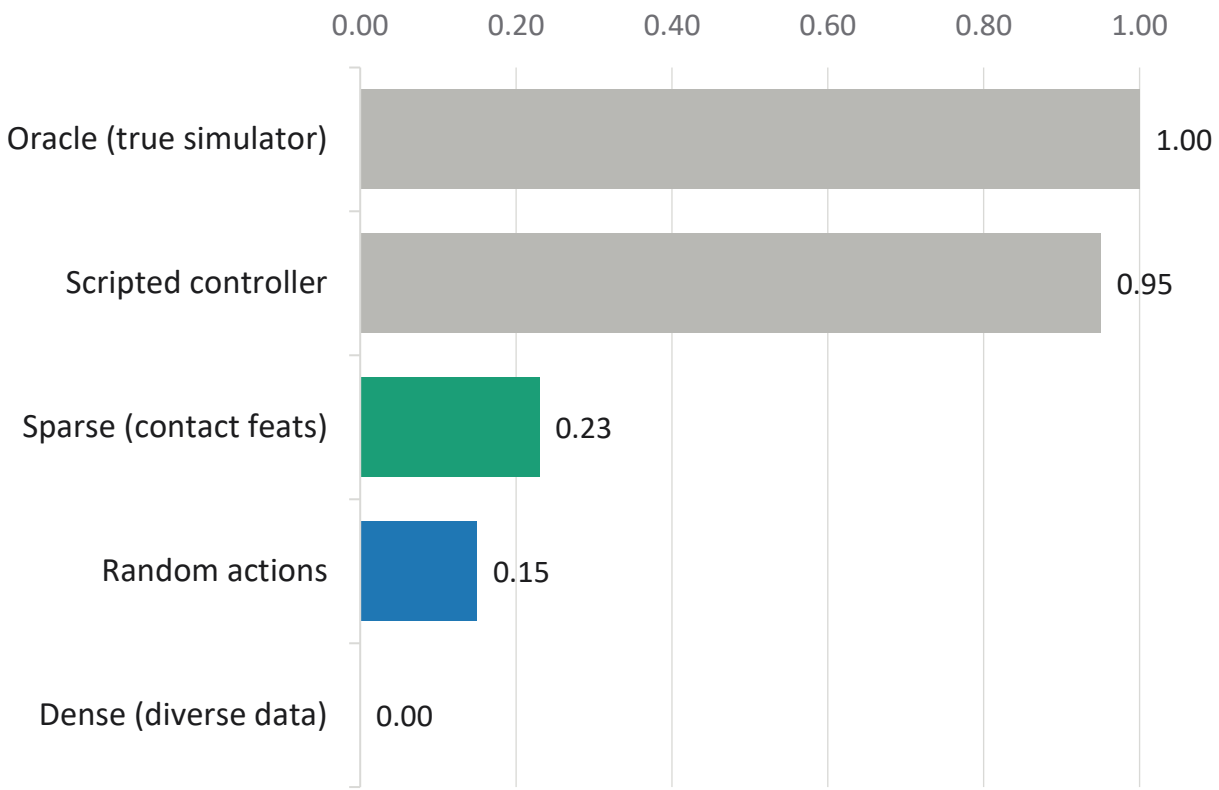
One checkpoint, any object count. Off-diagonal F1 0.827 vs 0.832 on the diagonal: 99.4% retention. Dense cannot even be run off-count.

(c) Sample efficiency

Sparse reaches about 90% of its full-data F1 with 25% of the data. Dense is flat at every budget.

DOWNSTREAM PLANNING

Can it plan? Separated from dense, not yet from random



Push-target-to-goal, 3 objects, success = target within 5 cm of goal. CEM planner (256 samples × 3 iterations, horizon 15, replan every step); only the model is swapped. Learned models: mean of 3 seeds × 20 episodes (Table IV).

Round 1: prediction-trained models cannot plan

Both learned models: 0.00, worse than random. The planner visits teleported, near-rest states far from the training data. Dense hallucinates 5–10 cm of motion with no contact; the sparse gate goes silent.

Round 2: fix the diagnosed cause

Contact-aware, velocity-free features and diverse mixed-policy training data. Same planner. Sparse goes 0.00 → 0.23 ± 0.06 and halves final distance. Dense stays at 0.00 at every seed.

Robust finding: separation from dense at every seed. Not robust: the margin over random (weakest seed is exactly 0.15).

READING THE RESULTS HONESTLY

What the headline is, and what it is not

IT IS

- **A change detector that avoids error injection:** F1 0.80–0.87 where dense is degenerate; unchanged-object error 0.211 → 0.001
- **Far better than dense** one-step (2.5–4.6×), in rollout (3.4–6.4×), and in planning (0.23 vs 0.00 at every seed)
- **Count-invariant and sample-efficient,** with an inspectable per-object change mask
- **8.6–11.1× fewer parameters**

IT IS NOT

- **An accurate mover predictor:** changed-object L2 sits at or barely above no-op; overall L2 only matches no-op at N = 5, 8
- **Better than no-op in rollout:** worse than no-op overall at every N and horizon
- **Significantly better than random at planning:** 0.23 ± 0.06 vs 0.15, one planner, one task, 3 objects
- **Faster in wall-clock,** or pixel-space: structured state only

CONCLUSION

Takeaways

1 Model what changes.

A per-object gate plus a masked residual is a one-line bias that stops a world model from corrupting the static majority of a scene.

2 Decompose the metric.

Overall L2 hid the story. Splitting error into changed and unchanged objects, and always reporting no-op, is what made the claim honest.

3 The delta head is the lever.

Detection is solved to F1 0.8+; regression of contact deltas is not. Next: a dense-residual baseline, a distributional delta head, DAgger on planner states, multi-step training, slot-based encoders.

Thank you. Questions welcome.

Code, data generators, checkpoints, result tables

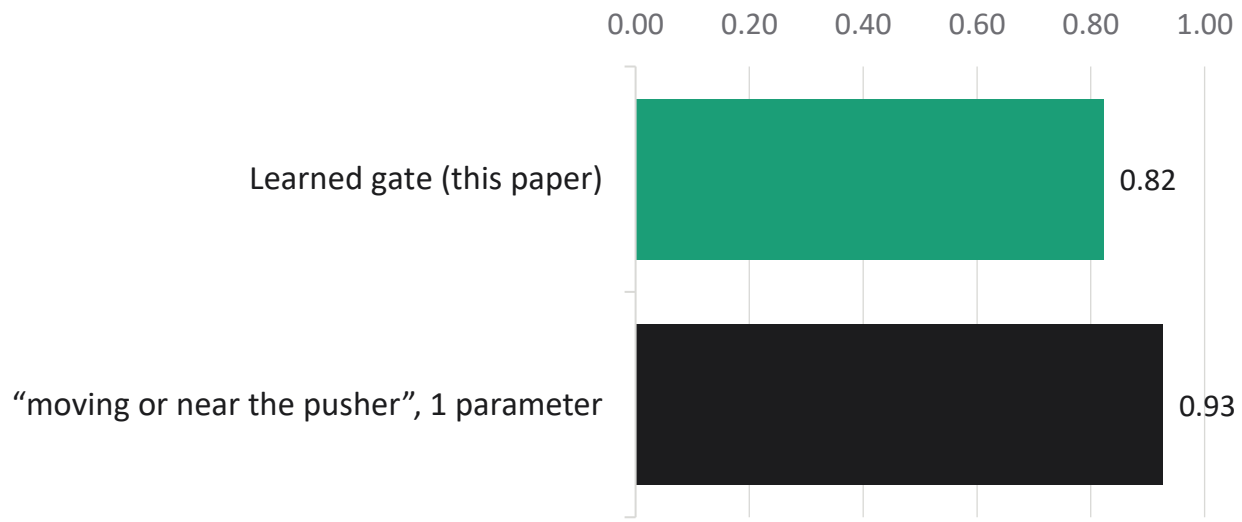
github.com/ParamThakkar123/sparse_world_models

Live in-browser demo runs the real checkpoints

POSTSCRIPT (OPTIONAL)

Since the camera-ready: run the trivial baselines

We asked what a one-parameter rule scores on our own change-detection benchmark. It beats the learned gate.



Motion-filtered benchmark, 3 objects, F1 on held-out transitions.

Follow-up work in preparation. Audit battery and benchmarks in the repository.

- With one point-like pusher, **the changed label is recoverable from a single scalar:** speed, or distance to the pusher
- Filtering the data harder just changes *which* one-liner wins
- The error-suppression mechanism stands; the **benchmark** does not. Every “our model predicts what changes” claim needs a trivial-baseline audit

APPENDIX C

Backup: efficiency in full

N	sparse params	dense params	param ratio	sparse FLOPs	dense FLOPs	latency ms (sparse / dense)
3	6 916	76 809	11.1×	39 936	152 576	0.19 / 0.04
5	8 452	82 959	9.8×	81 920	164 864	0.20 / 0.04
8	10 756	92 184	8.6×	167 936	183 296	0.20 / 0.04

Parameter efficiency is the durable win. The operation-count advantage erodes from 3.8× to 1.1× as per-object heads scale with N, and the dense model's single matmul is faster in CPU latency. We claim capacity efficiency, not speed.

APPENDICES D AND G

Backup: sparsity weight and per-seed planning

Sparsity weight λs (3 objects, seed 0)

λs	F1	precision	recall	changed L2	overall L2
0.0	0.873	0.941	0.814	0.389	0.146
0.05	0.865	0.940	0.801	0.389	0.146
0.2	0.849	0.938	0.776	0.388	0.145
0.5	0.845	0.967	0.750	0.387	0.144
1.0	0.815	0.965	0.705	0.385	0.143

Contact-aware planning, per seed

model	seed 0	seed 1	seed 2	mean ± std
sparse success	0.25	0.15	0.30	0.23 ± 0.06
dense success	0.00	0.00	0.00	0.00 ± 0.00
sparse final dist.	0.188	0.231	0.193	0.204 ± 0.019

Parameter-matched dense (Appendix E): shrinking dense to the sparse budget gives L2 0.363 / 0.363 / 0.415 and F1 0.538 / 0.388 / 0.294 at N = 3 / 5 / 8. Sparse still wins on every metric at every count, so the degeneracy is not a capacity artifact.